



# LUMINA PRO

Application Kanban de gestion de tâches

**SYRINE BEN HASSINE**

# SOMMAIRE

**1** Présentation personnelle

**2** Reconversion professionnelle

**3** Pourquoi ce projet ?

**4** État du marché du Kanban

**5** Projet solo — justification

**6** Cahier des charges

**7** Fonctionnalités

**8** User stories

**9** Cas d'utilisation

**10** Démo Kanban

**11** Choix des technologies

**12** Architecture applicative

**13** Planning

**14** Moodboard & charte graphique

**15** Wireframes & maquettes

**16** Intégration HTML/CSS

**17** Responsive

**18** Accessibilité, SEO & Lighthouse

**19** Base de données — MCD/MLD/MPD/SQL

**20** Authentification JWT

**21** Liste des routes

**22** Swagger

**23** Sécurité

**24** Cahier de tests

**25** Tests

**26** Déploiement

**27** Améliorations futures

**28** Conclusion

# PRÉSENTATION PERSONNELLE

## Qui suis-je ?

---

Passionnée de mode depuis toute petite, je suis diplômée en stylisme-modélisme et j'ai obtenu une certification en informatique et bureautique en 2001. J'ai ensuite été gérante d'ateliers de couture, où je réalisais déjà mon stylisme sur ordinateur. J'ai toujours été fascinée par les nouvelles technologies et leurs mystères — de nature très curieuse, j'ai voulu faire du code pour découvrir la face cachée derrière les belles vitrines des sites web : comment elles sont vraiment conçues.

**Formation :** DWWM — Développeuse Web et Web Mobile · La Plateforme\_ · Promotion 2025/2026

# RECONVERSION PROFESSIONNELLE

## Pourquoi ce changement de voie ?

---

Avant cette formation, je gérais des ateliers de couture et je réalisais du stylisme assisté par ordinateur. La passion pour la technologie n'a jamais disparu depuis ma première certification en bureautique en 2001 : en découvrant la formation de La Plateforme\_, j'ai voulu la poursuivre pour acquérir de nouvelles connaissances et avancer vers ce monde de la technologie qui façonne l'avenir. C'est cette curiosité naturelle pour comprendre ce qui se cache derrière un site web qui m'a menée au code.

Une reconversion est un atout, pas une faiblesse — elle montre de la motivation et de la maturité de choix.

# POURQUOI CE PROJET ?

## D'où vient l'idée de Lumina Pro ?

---

L'idée de départ : beaucoup d'équipes perdent du temps à suivre leurs tâches sur des supports dispersés (messages, tableurs, post-its). Lumina Pro répond à un besoin simple — **tout centraliser dans un seul tableau visuel**, sans la complexité d'un outil comme Jira.

Le nom et la signature du projet résument cette intention :

*« Organisez vos idées avec clarté »*

**L'anecdote derrière le choix :** pendant le projet de groupe MarsIA, nous avons utilisé Trello pour organiser le travail d'équipe — j'ai beaucoup aimé ses fonctionnalités et la facilité avec laquelle il permettait de suivre l'avancement de chacun. C'est cette expérience concrète qui m'a donné envie de comprendre et de recréer moi-même un outil Kanban, de A à Z.

# ÉTAT DU MARCHÉ DU KANBAN

## Des outils puissants, mais souvent lourds

### TRELLO

Simple et populaire. Utilisé pendant ce projet pour le suivi des fonctionnalités à développer.

### MONDAY.COM

Outil de gestion de projet tout-en-un, très visuel, mais coûteux et surdimensionné pour une petite équipe.

### JIRA

Très complet pour les équipes agiles, mais complexe à configurer et coûteux pour une petite structure.

### ASANA

Riche en fonctionnalités (rapports, automatisations), au prix d'une prise en main plus longue.

**Positionnement de Lumina Pro :** un Kanban resserré sur l'essentiel, pensé comme outil de certification — pas une alternative commerciale à ces solutions.  
L'objectif : prouver la maîtrise de toute la chaîne (BDD → API → sécurité → interface).

🕒 Durée de réalisation : **3 mois** (mai → juillet 2026).

# PROJET SOLO

## Pourquoi seule plutôt qu'en équipe ?

---

- ▶ Le référentiel DWWM autorise et évalue un projet individuel
- ▶ Travailler seule oblige à comprendre **toute la chaîne** (BDD, back-end, front-end, sécurité) plutôt qu'une seule partie
- ▶ Pas de dépendance à la disponibilité d'un binôme sur le temps de formation

**En contrepartie :** un périmètre volontairement resserré (MVP), pour livrer un projet complet et solide plutôt qu'un projet ambitieux mais inachevé.

# CAHIER DES CHARGES

## Fonctionnalités clés

---

- ▶ **Tableau Kanban** : visualisation des tâches par colonnes (À faire, En cours, Terminé)
- ▶ **Système d'authentification** : accès sécurisé par email / mot de passe
- ▶ **Gestion des rôles** : distinction entre administrateur et utilisateur
- ▶ **Interface responsive** : utilisable aussi bien sur desktop que sur mobile

# FONCTIONNALITÉS

## Fonctionnalités retenues (MVP livré)

---

- Tableau Kanban, colonnes personnalisables
- CRUD complet des tâches
- Authentification JWT + bcrypt
- Rôles admin / utilisateur
- Documentation API Swagger

### **ADMIN**

Gère les colonnes et les utilisateurs, peut supprimer n'importe quelle tâche

### **UTILISATEUR**

Crée et déplace des tâches, ne peut supprimer que ses propres tâches assignées

# USER STORIES

## Besoins exprimés par profil

---

### EN TANT QU'UTILISATEUR, JE PEUX...

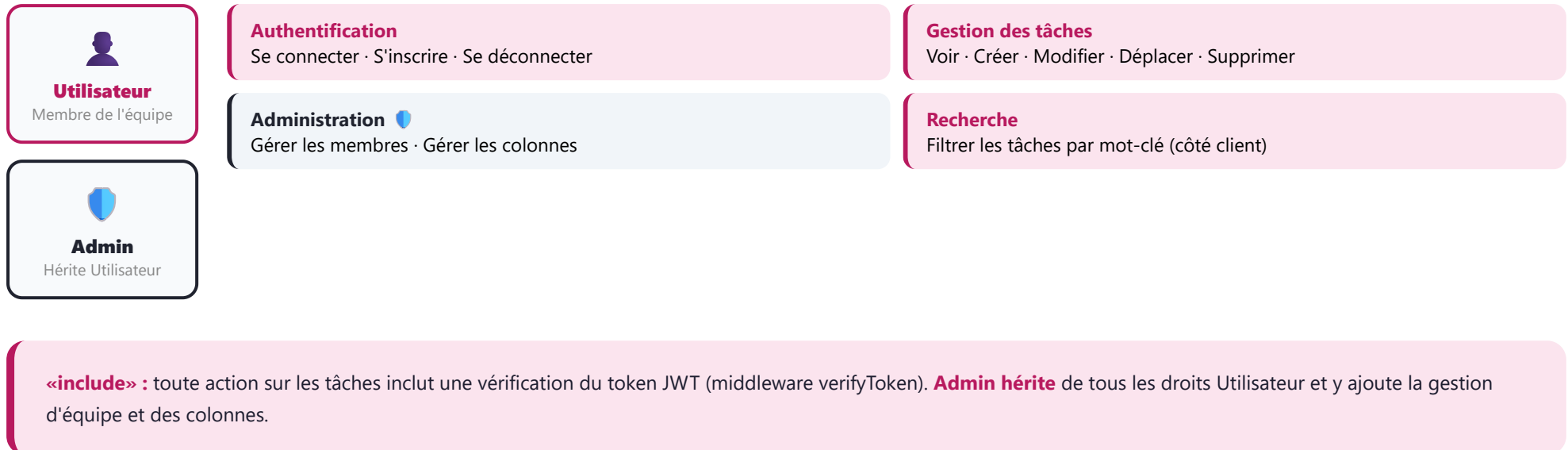
- **Authentification** : me connecter et m'inscrire de façon sécurisée
- **Tâches** : voir le tableau Kanban de l'équipe, créer et assigner des tâches, les déplacer entre colonnes, modifier ou supprimer mes propres tâches
- **Équipe** : consulter la liste des membres
- **Suivi** : consulter les statistiques du projet

### EN TANT QU'ADMIN, JE PEUX FAIRE TOUT ÇA, PLUS...

- **Tâches** : supprimer n'importe quelle tâche de l'équipe, pas seulement les miennes
- **Colonnes** : créer, renommer et supprimer une colonne du tableau
- **Équipe** : voir et retirer un membre de l'équipe

# DIAGRAMME DE CAS D'UTILISATION

## UML — 2 acteurs, 14 cas d'utilisation



# KANBAN

## Génération dynamique du tableau

```
// frontend/js/kanban.js - renderBoard()
function renderBoard(columns, tasks) {
  board.innerHTML = columns.map(col => {
    const colTasks = tasks.filter(t => t.id_col === col.id);
    return `<div class="kanban-column"> ... ${colTasks.map(t => renderTask(t)).join('')} ... </div>`;
  }).join('');
}
```

**Flux :** vérif token → fetchTasks() → 2 appels API (colonnes + tâches) → renderBoard() → filter/map/join.

# CHOIX DES TECHNOLOGIES

## ⚡ FRONT-END

- ▶ HTML5 / CSS3 / JS ES6+ vanilla
- ▶ Pas de framework : comprendre chaque ligne

## 🔧 BACK-END

- ▶ Node.js v24.14.0 + Express 5.2.1
- ▶ jsonwebtoken 9.0.3 + bcrypt 6.0.0 pour la sécurité
- ▶ dotenv 17.4.2 (secrets en .env), cors 2.8.6

## 🗄️ BASE DE DONNÉES

- ▶ **MySQL 8.0.45** — relationnel, adapté à des données liées (tâches ↔ utilisateurs ↔ colonnes)
- ▶ mysql2 3.22.2 comme driver Node

## 🔧 OUTILS

- ▶ Swagger UI (swagger-ui-express 5.0.1), Git/GitHub, XAMPP, VS Code

**Justification du choix "vanilla" :** objectif pédagogique — être capable d'expliquer chaque mécanisme (fetch, DOM, JWT) sans dépendre d'un framework qui ferait le travail à ma place.

# ARCHITECTURE APPLICATIVE

## Un MVC allégé



- **Vue** : pages HTML statiques (login.html, index.html) + modules JS qui manipulent le DOM
- **Contrôleur** : chaque fichier de backend/routes/ reçoit la requête HTTP et orchestre la réponse
- **Modèle** : pas de couche modèle séparée — chaque route exécute directement ses requêtes SQL via `req.db.query()`

Sur un projet solo de cette taille (4 tables), une séparation controller/model stricte aurait ajouté de l'indirection sans bénéfice réel. Cette simplification est identifiée comme première évolution si l'application devait grandir (voir roadmap).

# PLANNING — DIAGRAMME DE GANTT

## Reconstitué depuis l'historique Git réel (mai → juillet 2026)



Suivi du projet via **Trello** (tableau Kanban des fonctionnalités à développer) et **33 commits Git** qui retracent l'historique réel du développement. Planning reconstitué a posteriori à partir de ces commits plutôt que d'un GANTT prévisionnel théorique.

# MOODBOARD & INSPIRATIONS UX/UI

## Références visuelles ayant guidé la charte

---

- ▶ Recherche de composants Kanban sur **Dribbble** pour la mise en page des colonnes et cartes de tâches
- ▶ Palette de marque en dégradés rose/fuchsia (#B8195E), choisie pour rester sobre tout en étant reconnaissable
- ▶ Sidebar de navigation sombre + zone de contenu claire : un schéma courant des outils SaaS de gestion, pour une prise en main immédiate
- ▶ Priorité donnée à la lisibilité : peu de couleurs, un code couleur unique et cohérent pour les priorités (rouge/ambre/vert)

Ces références (Dribbble + codes visuels SaaS) tiennent lieu de moodboard : pas de planche d'inspiration formelle constituée en amont, mais une recherche visuelle ciblée qui a directement nourri la charte graphique (couleurs, typographie, structure sidebar/contenu).

# CHARTE GRAPHIQUE

## Pourquoi ces choix ?

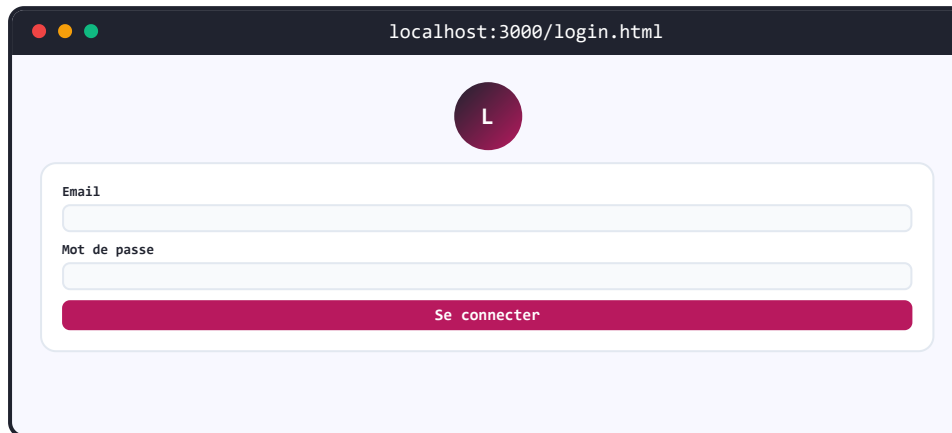
---

- ▶ **Rose/fuchsia** : couleur de marque reconnaissable, associée à l'idée de créativité, tout en restant sobre sur le reste de l'interface
- ▶ **Sidebar sombre + contenu clair** : sépare clairement la navigation du contenu de travail, repère visuel immédiat
- ▶ **Peu de couleurs au total** : évite la surcharge visuelle, chaque couleur garde un sens précis (priorité, action, navigation)
- ▶ **Police système (Inter)** : pas de chargement externe, cohérence garantie sur toutes les machines

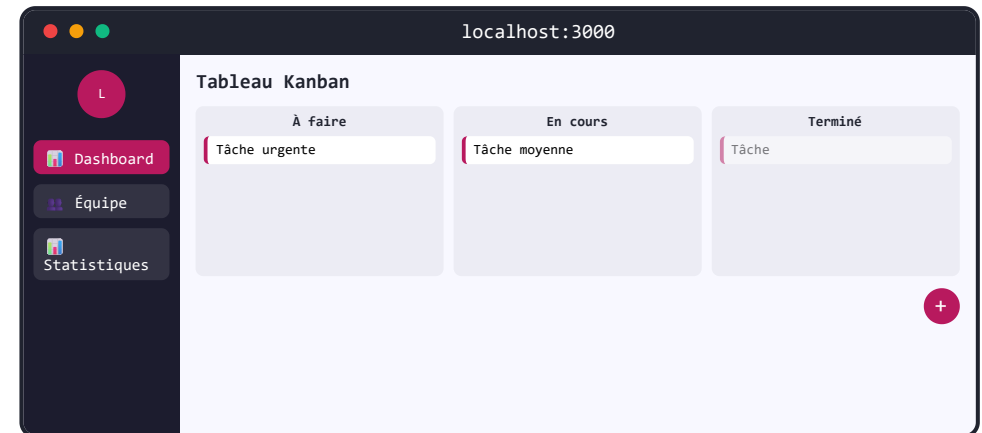
# WIREFRAMES & MAQUETTES

## Maquettes basse fidélité

login.html — Connexion / Inscription



index.html — Vue Kanban (dashboard protégé JWT)



Wireframes basse fidélité réalisés en amont pour valider la navigation avant tout travail graphique — détail complet (arborescence, tous les écrans) dans `arborescence_wireframes.html`.

# INTÉGRATION FRONT HTML/CSS

## Une base HTML/CSS modulaire

---

- ▶ HTML5 sémantique (header, nav, main)
- ▶ CSS3 découpé en **8 fichiers** par responsabilité (variables, layout, kanban, responsive, formulaires, composants...)
- ▶ Flexbox pour les mises en page, media queries pour le responsive
- ▶ Aucun framework CSS : chaque règle est écrite et comprise à la main

### authFetch() — centraliser les appels réseau

```
// frontend/js/api.js
async function authFetch(url, options = {}) {
  const token = localStorage.getItem('token');
  options.headers = { 'Authorization': `Bearer ${token}`, ...options.headers };
  const response = await fetch(url, options);
  if (response.status === 401) { localStorage.clear(); window.location.href = 'login.html'; }
  return response;
}
```

# RESPONSIVE

## Adaptation mobile avec CSS

### Desktop

- ▶ Sidebar fixe à gauche
- ▶ Colonnes kanban côte à côte

### Tablette ( $\leq 900\text{px}$ )

- ▶ Sidebar en tiroir (masquée par défaut), bouton hamburger ☰
- ▶ Colonnes kanban toujours côte à côte, largeur fixe (270px), scroll horizontal si besoin

### Mobile ( $\leq 600\text{px}$ )

- ▶ Sidebar en tiroir plein écran
- ▶ Colonnes empilées verticalement (scroll vertical, plus horizontal)

```
/* css/responsive.css */
@media (max-width: 900px) {
  .sidebar { left: -280px; }
  .sidebar.open { left: 0; }
  .kanban-column { min-width: 270px; width: 270px; }
}
@media (max-width: 600px) {
  .kanban-board { flex-direction: column; overflow-x: hidden; }
  .kanban-column { width: 100%; min-width: 100%; }
}
```

# ACCESSIBILITÉ, SEO & Lighthouse

## Accessibilité (RGAA / WCAG 2.1)

- ▶ `aria-label` sur les boutons icônes (sans texte visible)
- ▶ Navigation clavier complète, focus visible sur chaque élément interactif
- ▶ Contrastes vérifiés, `<label>` associé à chaque champ de formulaire
- ▶ `aria-live="polite"` sur les zones qui changent dynamiquement (liste des tâches)

## SEO

- ▶ HTML5 sémantique (header, nav, main), un seul `<h1>` par page
- ▶ `<strong>` pour marquer les mots-clés importants dans le texte (le moteur de recherche leur accorde plus de poids qu'à du texte normal, sans changer le sens visuel — c'est du gras sémantique, pas juste stylistique)
- ▶ `<meta name="description">` sur chaque page pour le référencement (extrait affiché dans les résultats de recherche)
- ▶ Référencement naturel limité par nature : l'application est derrière une authentification, donc peu indexable au-delà de la page de connexion

## Audit Lighthouse — page index.html

### PERFORMANCE

97

### ACCESSIBILITÉ

96

### BEST PRACTICES

100

### SEO

100

**Performance à 97** : chargement quasi instantané (First Contentful Paint 0,3 s, Total Blocking Time 0 ms) — seule piste d'amélioration : optimiser le poids des images (42 KiB)

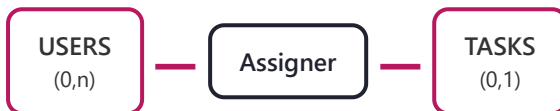
# MCD — MODÈLE CONCEPTUEL DE DONNÉES

## Entités, associations, cardinalités (méthode MERISE)

| COLUMNS  |
|----------|
| id       |
| title    |
| position |

| TASKS       |
|-------------|
| id          |
| title       |
| description |
| priority    |
| created_at  |

| USERS    |
|----------|
| id       |
| name     |
| email    |
| password |
| role     |



Une colonne contient 0 à n tâches, une tâche appartient toujours à exactement une colonne (1,1). Un utilisateur peut se voir assigner 0 à n tâches ; une tâche est assignée à 0 ou 1 utilisateur (assignation optionnelle). Les deux relations portent un verbe à l'infinitif, conformément à la notation MERISE.

# MLD — MODÈLE LOGIQUE DE DONNÉES

## Traduction en tables, indépendante du SGBD

```
USERS (id PK, name, email, password, role)
COLUMNS (id PK, title, position)
TASKS (id PK, title, description, priority, created_at,
       id_col FK → COLUMNS.id,
       id_assigned FK → USERS.id)
```

Le MLD ne porte plus de cardinalités : chaque association 1,n devient une clé étrangère placée du côté "n". TASKS.id\_col matérialise la relation Contenir, TASKS.id\_assigned matérialise la relation Assigner (nullable, car l'assignation est optionnelle).

# MPD — MODÈLE PHYSIQUE DE DONNÉES

## Implémentation MySQL (database/schema.sql)

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  role ENUM('admin', 'user') DEFAULT 'user'  
);  
  
CREATE TABLE columns (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  position INT DEFAULT 0  
);  
  
CREATE TABLE tasks (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(255) NOT NULL, description TEXT,  
  priority ENUM('high', 'medium', 'low') DEFAULT 'medium',  
  id_assigned INT, id_col INT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_assigned) REFERENCES users(id) ON DELETE SET NULL,  
  FOREIGN KEY (id_col) REFERENCES columns(id) ON DELETE CASCADE  
);
```

SGBD réel : **MySQL 8.0.45**. Le MPD ne contient pas de cardinalités — uniquement des types de colonnes et des contraintes.

# SQL — EXEMPLES CRUD

## Un Create, un Read, un Update, un Delete

```
// 1. CREATE - routes/auth.js, à l'inscription
const sql = "INSERT INTO users (name, email, password) VALUES (?, ?, ?)";

// 2. READ - routes/tasks.js, liste des tâches
const sql = "SELECT * FROM tasks";

// 3. UPDATE - routes/tasks.js, construction dynamique selon les champs reçus
let sql = "UPDATE tasks SET "; if (title) sql += "title = ?, "; ... + " WHERE id = ?";

// 4. DELETE - la suppression d'une colonne entraîne celle de ses tâches
const sql = "DELETE FROM columns WHERE id = ?"; // ON DELETE CASCADE supprime ses tâches
```

Toutes les requêtes utilisent des placeholders ? (requêtes préparées) — jamais de concaténation directe de variables dans le SQL.

# AUTHENTICATION JWT

## JSON Web Token — flux complet



```
// middleware/security.js
function verifyToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  if (!authHeader?.startsWith('Bearer '))
    return res.status(401).json({ error: 'Token manquant' });
  try {
    req.user = jwt.verify(authHeader.split(' ')[1], SECRET_KEY);
    next();
  } catch { res.status(403).json({ error: 'Token invalide' }); }
}
```

# LISTE DES ROUTES

## API REST — server.js

```
app.use('/api/auth', require('./routes/auth'));
app.use('/api/tasks', verifyToken, require('./routes/tasks'));
app.use('/api/columns', verifyToken, require('./routes/columns'));
app.use('/api/users', verifyToken, require('./routes/users'));
```

| Méthode    | Route                                | Accès                       |
|------------|--------------------------------------|-----------------------------|
| POST       | /api/auth/login, /register           | Public                      |
| GET        | /api/tasks, /api/columns, /api/users | JWT                         |
| POST / PUT | /api/tasks, /api/columns             | JWT                         |
| DELETE     | /api/tasks/:id                       | JWT — propriétaire ou admin |
| DELETE     | /api/columns/:id, /api/users/:id     | JWT                         |

# SWAGGER

**URL :** <http://localhost:3000/api-docs>

Documentation générée depuis `swagger.json` : chaque route y est décrite avec ses paramètres, le corps de requête attendu et ses codes de retour.

Remplace l'usage habituel de Postman — documentation et test au même endroit, versionnés avec le code.

```
{
  "paths": {
    "/auth/login": {
      "post": {
        "summary": "Connexion d'un utilisateur",
        "tags": ["Authentification"],
        "security": [],
        "requestBody": {
          "required": true,
          "content": {
            "application/json": {
              "schema": {
                "type": "object",
                "properties": {
                  "email": { "type": "string" },
                  "password": { "type": "string" }
                }
              }
            }
          }
        },
        "responses": {
          "200": { "description": "Succès - Retourne un token JWT" },
          "401": { "description": "Identifiants incorrects" }
        }
      }
    }
  }
}
```

# SÉCURITÉ



## Injection SQL → requêtes préparées

Placeholders ? sur toutes les requêtes — jamais de concaténation directe.



## XSS → escapeHTML()

Tout contenu utilisateur affiché dans le DOM est échappé avant insertion.



## Mots de passe → bcrypt (10 rounds)

`bcrypt.hash(password, 10)` à l'inscription, `bcrypt.compare()` à la connexion — jamais stocké en clair.



## CSRF / CORS / RGPD

Token JWT en header (jamais en cookie) ; CORS activé (à restreindre en prod) ; données personnelles minimales, jamais stockées en clair.



## Suppression restreinte au propriétaire

Un utilisateur ne peut supprimer que ses propres tâches ; seul l'admin peut tout supprimer — vérifié côté serveur.

# CAHIER DE TESTS

## 12 scénarios manuels exécutés contre le serveur réel

| Test    | Description                                             | Résultat |
|---------|---------------------------------------------------------|----------|
| TEST001 | Accessibilité du site                                   | OK       |
| TEST002 | Connexion avec un compte valide                         | OK       |
| TEST003 | Impossible de se connecter avec un compte inexistant    | OK       |
| TEST004 | Accès à une route protégée sans token                   | OK       |
| TEST005 | Création d'une tâche sans titre                         | OK       |
| TEST006 | Création d'une tâche valide                             | OK       |
| TEST007 | Liste des colonnes                                      | OK       |
| TEST008 | Déplacement d'une tâche entre colonnes                  | OK       |
| TEST009 | Suppression d'une colonne contenant une tâche (cascade) | OK       |
| TEST010 | Liste des utilisateurs sans exposition du mot de passe  | OK       |
| TEST011 | Inscription d'un nouveau compte                         | OK       |
| TEST012 | Connexion avec le compte fraîchement inscrit            | OK       |

Détail complet dans [revision\\_diplôme/Cahier\\_de\\_Tests\\_Lumina\\_Pro.html](#). Aucun test automatisé (Jest/Supertest) n'est encore en place — identifié en amélioration future.

# TESTS

## Vérifier avant de livrer

---

### TESTS MANUELS

- Parcours utilisateur (login, CRUD, déplacement)
- Rôles admin / utilisateur testés séparément

### OUTILS

- Swagger UI, DevTools

### AUTOMATISÉS

- Non implémentés — piste : Jest + Supertest

# DÉPLOIEMENT

## Du poste local à la mise en ligne

### LOCAL

- MySQL 8.0.45 + `node server.js`
- Démarrage manuel : service MySQL80 puis `npm start`

### TEST

- Variables dans `.env`
- Comptes de démo créés via `/api/auth/register`

### PRODUCTION (PISTE)

- Hébergement mutualisé/VPS, HTTPS

Aujourd'hui, le projet tourne en local (XAMPP + Node.js). La mise en ligne est une prochaine étape, notée dans la roadmap.

# AMÉLIORATIONS FUTURES

## Ce que j'améliorerais en production

---

### SÉCURITÉ

- HTTPS obligatoire
- Rate limiting anti-bruteforce sur Redis (déjà en place en mémoire)

### QUALITÉ

- Tests automatisés Jest
- Validations backend renforcées

### FONCTIONNALITÉS

- Drag & drop natif
- Docker pour le déploiement

# CONCLUSION

Lumina Pro m'a permis de mettre en pratique l'ensemble des compétences du titre DWWM :

- ✓ **Front-end** : HTML/CSS responsive, JavaScript modulaire, Fetch API
- ✓ **Back-end** : API REST Node.js/Express, authentification JWT, hachage bcrypt
- ✓ **Base de données** : conception MCD/MLD/MPD, SQL, relations
- ✓ **Sécurité** : injection SQL, XSS, mots de passe, autorisation par propriétaire

C'est un bon début pour un projet que je compte continuer à faire vivre au-delà de l'examen. Pour qu'il se distingue vraiment, les prochaines étapes seraient : le drag & drop natif pour une expérience plus fluide, des tests automatisés pour sécuriser les évolutions futures, et une vraie mise en production accessible en ligne — plutôt que d'ajouter des fonctionnalités superflues, je préfère consolider ce cœur applicatif avant de l'étendre.



**Merci pour votre attention**

— Questions ? —



# Lumina Pro

TABLEAU KANBAN · APPLICATION WEB SÉCURISÉE

**Syrine Ben Hassine**

syrine.ben-hassine@laplateforme.io

**Merci pour votre attention**

— Questions ? —